kata – Describe katas with pictograms

Lukas Heindl*

0.1.0 from 2026-03-23

Abstract

The `kata` package provides a compact and structured way to typeset martial arts katas[1] (forms) in LaTeX. It models a kata as a sequence of steps, each combining the stance hand/foot technique, and transition direction, and supports customizing the resulting pictograms for clear visual representation.

Note that this in no way is a standardized or agreed upon way to document such forms (to my the best of my knowledge nothing agreed upon exists). This is just a presentation I came up with (it's not that complicated of course) and which works for me.

1 Introduction

In many martial arts such as Karate, a kata is a predefined sequence of movements[2].

The `kata` package aims to provide a simple, structured way to describe katas in LaTeX. Each step of a kata is expressed as a step consisting of

1. the stance with direction
2. a technique with direction
3. the direction of movement or rotation into the next step

The package also offers customizing the resulting pictograms, allowing visual annotation of sequences, such as highlighting repetitions or emphasizing specific actions.

Note that this in no way is a standardized or agreed upon way to document such forms (to my the best of my knowledge nothing agreed upon exists). This is just a presentation I came up with (it's not that complicated of course) and which works for me.

*E-Mail: oss.heindl+latex@protonmail.com
Issue tracker: codeberg.org/atticus-sullivan/kata/issues
Github (mirror): github.com/atticus-sullivan/kata
Codeberg (mirror): codeberg.org/atticus-sullivan/kata

My gratitude goes to Clea F. Rees who provided valuable advice in general and in particular helped in enhancing the code quality. Also she helped a lot in not only making the `memoize` integration work, but also make it play well with other memoizable content.

[1] https://en.wikipedia.org/wiki/Kata
[2] https://en.wikipedia.org/wiki/Kata

2 Usage / Quick-Start

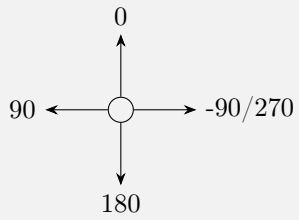
Load this package as `\usepackage[<options>]{kata}`.

This section documents every command and environment which mostly is all you need to know to start using this package. Together with the simple example (see section 2.3) this should get you working.

Then, you can look up customization options later in section 2.4 (either by reading everything or following the links embedded in this section).

Note: Orientation of the *angles* in the arguments

Some commands such as `\kataWaza`→P. 4, `\kataDir`→P. 4, and `\kataStance`→P. 4 can/must be passed angles in the range of -360 to 360 degrees. Also some pgfkeys take a angles as an argument.
This package consistently defines the orientation of these angles as follows:



2.1 Package arguments

This package supports passing package-options.

See section 2.4 for the available ⟨*options*⟩. Note, there will be issues when spaces occur in options (the key name). Thus, often you will have to set the options with `\kataSetup` instead.

`\kataSetup[⟨options⟩]`

This can set exactly the same options like with the package-options. You can either use this to set package-options at a later point in time in the preamble. But you can also use this to set options mid-document. Note that these options are then local to that scope. You can leverage this to change the style of a sequence of `kataStep`→P. 3s "globally" (see also section 2.5).

`\kataAliases`

Defines the following aliases for ease of use:

`\stance[⟨side⟩]{⟨content⟩}{⟨dir⟩}[⟨options⟩]`

an alias for `\kataStance`→P. 4

`\waza[⟨side⟩]{⟨content⟩}{⟨dir⟩}[⟨options⟩]`

an alias for `\kataWaza`→P. 4

`\dir[⟨options⟩]{⟨angleA⟩}{⟨angleB⟩}`

an alias for `\kataDir`→P. 4

`\kiai{⟨angle⟩}`

an alias for `\kataKiai`→P. 5

```
\begin{step}[⟨options⟩]
  ⟨environment content⟩
\end{step}
```

an alias for `kataStep`

Defining the aliases is deferred until the begin of the document. Each alias is only defined if the corresponding macro is not already defined (otherwise a warning is shown in the log)
At the moment, this option only has an effect when given in the preamble (via `\kataSetup`→ P. 2) or in the package-options.

2.2 Specifying a kata

```
\begin{kata}[⟨options⟩]
  ⟨environment content⟩
\end{kata}
```

The environment which holds an entire kata. Within it, normal LaTeX code can be typeset. Mostly the content usually consists of `kataStep` environments each representing one step in the kata.
Depending on the ⟨*options*⟩, this will also add various additional elements such as a section or a marginline.
See section 2.4 for the available ⟨*options*⟩.
Internally this environment uses `kataRaw`. Thus, the remarks on that environment are also relevant for this one (particularly regarding how the grid is formed and the relevance of `/kata/kata/columns`→ P. 9).

```
\begin{kataRaw}[⟨options⟩]
  ⟨environment content⟩
\end{kataRaw}
```

This is the lightweight variant of `kata`. Similarly it also holds an entire kata. The huge difference is it does *no additional styling of the kata*.
See section 2.4 for the available ⟨*options*⟩.
It just places the steps and tries to generatr a proper grid of `step`s. In order to do so, this environment automatically adds phantom/invisible `\rule`s in the end which are supposed to fill up the last line of `step`s. For this reason, it is crucial `/kata/kata/columns`→ P. 9 is set to the amount of `step`s fitting onto one line. This setting is used to determine in which column the current `step` shall be placed which is a requirement for the correct spacing. Also it is necessary in order to determine the amount of `\rule`s which need to be placed in the end.
Notes:

- You can set `/kata/kata/columns`→ P. 9 to `0` which *disables many automatisms*. As a result, between every `step` a simple `\hskip 0.0pt plus 1.0fil` is placed no matter what the current column is. Also, this disables the automatic placement of `\rule`s in the end (you may of course still padd manually with `\kataStepPhantom`→ P. 12s.

```
\begin{kataStep}[⟨options⟩]
  ⟨environment content⟩
\end{kataStep}
```

Draws a pictogram representing one step in the kata.
See section 2.4 for the available ⟨*options*⟩.
The body will be the content of a `tikzpicture` so normal LaTeX code won't work as expected. Instead, if you want/need to, you can draw arbitraty TikZ code in this environment.
Notes:

- The circle representing the *head*, is drawn right at the start of this environment as a node named *head*. You may use it as a regular TikZ node when writing custom TikZ-code. The advice is not to draw out of the normal bounds as this will break the alignment of the pictograms.
- The counter which is also normally shown in the top left corner of each step also adds a *reference point*. In effect, you can use `\label` to label a step and reference that step later with the typical LaTeX mechanisms.

Usually you'll use `\kataWaza`, `\kataStance` and `\kataDir` to populate the step with contents. Note that all of these commands may be present (multiple times) but they don't need to be present all.

`\kataWaza[`⟨*side*⟩`]{`⟨*content*⟩`}{`⟨*dir*⟩`}[`⟨*options*⟩`]`

Use this command to draw a technique with its direction.
Note: This command is quite similar to `\kataStance`.

⟨***side***⟩ may specify the side (R/L or right/left) which executes the technique (arbitrary text is allowed here)

⟨***content***⟩ the text describing the action

⟨***angle***⟩ the angle in which to execute the action. See section 2 for more details.

⟨***options***⟩ see section 2.4.6 for the available ⟨*options*⟩. The most common option probably here is `/kata/waza/flip` → P. 11.

`\kataStance[`⟨*side*⟩`]{`⟨*content*⟩`}{`⟨*dir*⟩`}[`⟨*options*⟩`]`

Use this command to draw the technique+direction for the stance.
Note: This command is quite similar to `\kataWaza`.

⟨***side***⟩ may specify the side (R/L or right/left) which executes the technique (arbitrary text is allowed here)

⟨***angle***⟩ the angle in which to execute the action. See section 2 for more details.

⟨***dir***⟩ the direction in which to execute the action

⟨***options***⟩ see section 2.4.6 for the available ⟨*options*⟩. The most common option probably here is `/kata/stance/flip` → P. 11.

`\kataDir[`⟨*options*⟩`]{`⟨*angleA*⟩`}{`⟨*angleB*⟩`}`

Use this command to draw an indicator indicating the direction/turn to transition to the next step.
See section 2 for more details on the notion of angles.

⟨***options***⟩ see section 2.4.7 for the available ⟨*options*⟩. The most common option probably here is `/kata/dir/flip` → P. 11.

⟨***angleA***⟩ is where the movement starts (in degrees -360 to 360)

⟨***angleB***⟩ is where the movement ends (in degrees -360 to 360)

Note for ⟨*angleA*⟩ the string *center* is also eligible. This causes the arrow to start at the center of the circle/head, indicating only a direction not a turn.

`\kataKiai{⟨angle⟩}`

Use this command to draw an indicator that this step involves *kiai*.
The ⟨*angle*⟩ specifies at which angle to place this indicator. This package does not keep track how full the pictogram already is. Thus, you need to manually check at which angle there is space for this indicator to be placed.

2.3 Simple Example

Note: The output looks slightly different in the example's output here. Usually, the section is a proper section and the marginline is really show in the margin.

Mocks are only used here in the docs avoid problems showing this inside the example box <*doc>

A simple example without customizations (but includes flip)

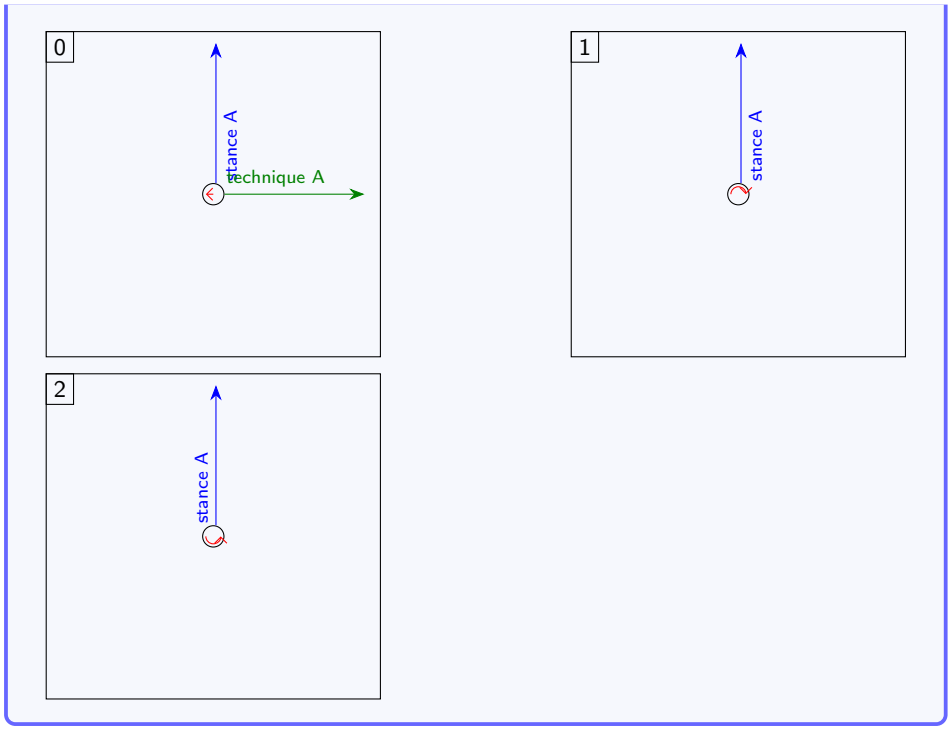
```
\kataSetup{styled kata={section,margin,belt},columns=2}

\begin{kata}[belt color=blue,title={Simple Example}]
  Some explanation -- normal \LaTeX{} code
  \parindent=0pt

  \begin{step}
    \stance{stance A}{0}
    \waza{technique A}{-90}
    \dir{center}{90}
  \end{step}
  \begin{step}
    \stance{stance A}{0}
    \dir{90}{-90}
  \end{step}
  \begin{step}
    \stance{stance A}{0}[flip]
    \dir[flip]{90}{-90}
  \end{step}
\end{kata}
```

X. Simple Example

Margin would be:
(*begin marginline*)

(*end marginline*)

Some explanation – normal LaTeX code



</doc>

`\kataStepCnt{`⟨*counter*⟩`}`

Formats the counter holding the number of the current step. Adjust this with e.g. `\RenewDocumentCommand` or `renewcommand` to change how the counter is displayed.

2.4 Options / Customization

The idea is you can set any option with any command (offering generic ⟨*options*⟩) which makes sense at this place. For example you can set the style in which the *border* of a step is drawn at the `kataStep`→P. 3, `kataRaw`→P. 3 and `\kataSetup`→P. 2/package level all with varying scopes. But you cannot set the *border* style at the `\kataWaza`→P. 4 or `\kataStance`→P. 4 level as it does not make sense.

Note: You should never needs to specify paths manually when specifying options. As long as you are only setting options which are valid for that command, it suffices to specify the last component of the path (e.g. when setting `/kata/stance/flip`→P. 11 in `\kataStance`→P. 4, it suffices to speficy the option as `flip`).

Structuring the options, it became evident there are groups of commands/environments which should accept a common set of options following that logic. These are described in section 2.4.1. Sections 2.4.2 to 2.4.7 describe options specific to the commands/environments.

2.4.1 Common options

These options can be applied at `kataStep`[→P. 3], `kataRaw`[→P. 3] and `\kataSetup`[→P. 2]/package level.

`/kata/setup/stance`=⟨*options*⟩
`/kata/kata/stance`=⟨*options*⟩
`/kata/step/stance`=⟨*options*⟩

Sets the ⟨*options*⟩ as if they are passed to every `\kataStance`[→P. 4] in the scope.
Scope means, e.g. if this is passed to `kataRaw`[→P. 3]/`kata`[→P. 3], all `\kataStance`[→P. 4] commands in this kata will get these ⟨*options*⟩. But for the next kata, the ⟨*options*⟩ are not applied.

`/kata/setup/waza`=⟨*options*⟩
`/kata/kata/waza`=⟨*options*⟩
`/kata/step/waza`=⟨*options*⟩

Similar like `/kata/setup/stance` etc.
Sets the ⟨*options*⟩ as if they are passed to every `\kataWaza`[→P. 4] in the scope.

`/kata/setup/dir`=⟨*options*⟩
`/kata/kata/dir`=⟨*options*⟩
`/kata/step/dir`=⟨*options*⟩

Similar like `/kata/setup/stance` etc.
Sets the ⟨*options*⟩ as if they are passed to every `\kataDir`[→P. 4] in the scope.

`/kata/setup/color`=⟨*options*⟩
`/kata/kata/color`=⟨*options*⟩
`/kata/step/color`=⟨*options*⟩

Use a list of the options documented in section 2.4.1 as ⟨*options*⟩. Allows adjusting the colors of the various elements which are drawn.
Scoping works similar like described for `/kata/setup/stance` etc.

`/kata/setup/head style`=⟨*options*⟩
`/kata/kata/head style`=⟨*options*⟩
`/kata/step/head style`=⟨*options*⟩

Appends to the syle used for the circle representing the *head*. You can specify arbitrary Ti*k*Z-code here.
Scoping works similar like described for `/kata/setup/stance` etc.

`/kata/setup/border style`=⟨*options*⟩
`/kata/kata/border style`=⟨*options*⟩
`/kata/step/border style`=⟨*options*⟩

Similar like `/kata/setup/head style` etc.
Appends the ⟨*options*⟩ to the style for the border drawn around each pictogram.

`/kata/setup/border preset`=⟨*presets*⟩
`/kata/kata/border preset`=⟨*presets*⟩
`/kata/step/border preset`=⟨*presets*⟩

Similar like `/kata/setup/head style` etc. But intead of appending raw Ti*k*Z options, this option allows passing ⟨*presets*⟩ defined by this package for the

border. See section 2.4.9 for more detailled information and a list of defined border presets.

`/kata/setup/step preset=`⟨*presets*⟩
`/kata/kata/step preset=`⟨*presets*⟩
`/kata/step/step preset=`⟨*presets*⟩

Similar like `/kata/setup/head style` → P. 7 etc. But intead of appending raw Ti*k*Z options, this option allows passing ⟨*presets*⟩ defined by this package for the steps. See section 2.4.9 for more detailled information and a list of defined step presets.

`/kata/setup/step counter style=`⟨*options*⟩
`/kata/kata/step counter style=`⟨*options*⟩
`/kata/step/step counter style=`⟨*options*⟩

Similar like `/kata/setup/head style` → P. 7 etc.
Appends the ⟨*options*⟩ to the style for the Ti*k*Z node showing the number of the step (usually placed at the top left corner).

Colors A collection of keys used to adjust the color used for drawing elements in the pictograms. As ⟨*color*⟩, everything can be passed here which Ti*k*Z can use as a color.

`/kata/color/stance=`⟨*color*⟩

The color used for the arrow showing the direction of the stance as well as the label on this arrow.

`/kata/color/waza=`⟨*color*⟩

The color used for the arrow showing the direction of the waza as well as the label on this arrow.

`/kata/color/dir=`⟨*color*⟩

The color used for the arrow showing the next direction/turn for transitioning to the next step.

`/kata/color/head=`⟨*color*⟩

The color used for the circle representing the head.

`/kata/color/border=`⟨*color*⟩

The color used for the border around the step.

`/kata/color/step counter=`⟨*color*⟩

The color used for the node (border + content) showing the number of the step.

2.4.2 Package/Setup specific

These options are specific to the package-options and `\kataSetup` → P. 2.

`/kata/setup/columns=`⟨*columns*⟩ (initially: 6)

Similar like `/kata/kata/columns` → P. 9.

`/kata/setup/definealiases` (cmd-key)

Defines the same aliases like `\kataAliases`[→P. 2] when set.
Defining the aliases is deferred until the begin of the document. Each alias is only defined if the corresponding macro is not already defined (otherwise a warning is shown in the log)
At the moment, this option only has an effect when given in the preamble (via `\kataSetup`[→P. 2]) or in the package-options.

`/kata/setup/styled kata`

Similar like `/kata/setup/stance`[→P. 7] etc.
Sets the ⟨*options*⟩ as if they are passed to every `kata`[→P. 3] (which is referred to as *styled kata* here in contrast to a *raw kata*) in the scope.

2.4.3 Raw Kata environment

These options are specific to `kataRaw`[→P. 3].

`/kata/kata/columns`=⟨*columns*⟩ (initially: 6)

Sets the amount of pictograms which fit one line in the document. As of now this package cannot measure this on its own so you need to set this manually (usually only once for the whole document via `/kata/setup/columns`[→P. 8]).
The effect of this is that at the end of a `kataRaw`[→P. 3], if the last line is not fully populated, it is filled up with `\rule`s with the width of one `step`[→P. 3].
In order to know if and if yes how many places need to be filled, this key is required. This is needed if you want a proper grid of pictograms in the output.
Note by setting this key to 0, you can deactivate this behavior.

2.4.4 Styled Kata environment

These options are specific to `kata`[→P. 3]. Note you can also pass keys from `/kata/kata` here (including common options section 2.4.1). As long as no key with the same name is defined here, the key (+value) will be forwarded to `/kata/kata`.

Section These keys are all about the sectioning.

`/kata/kata/styled/section` ((initially: false, default: true))

Determines whether to generate a section at all at the beginning of the `kata`[→P. 3].

`/kata/kata/styled/no section`

Shortcut for specifying `/kata/kata/styled/section=false`

`/kata/kata/styled/title`

The title/name of the kata. This is the content for the section (if it was enabled, see `/kata/kata/styled/section`).

`\kataSection`

The command with which the section is generated. The usual use-case of changing this key probably is to generate a *subsection* instead a *section* or similar changes.

Adjust it by using e.g. `\renewcommand[1]{\subsection{#1}}`.
Note, the specified macro must handle exactly one argument.

Margin These keys decide whether and if so what content to place in a marginline generated at the begin of the `kata`→P. 3.

`/kata/kata/styled/margin` ((initially: false, default: true))

Determines whether to place a marginline at all at the beginning of the `kata`→P. 3.

`/kata/kata/styled/no margin`

Shortcut for specifying `/kata/kata/styled/margin=false`

`/kata/kata/styled/belt` ((initially: false, default: true))

Determines if the note in the margin (if enabled, see `/kata/kata/styled/margin`) should contain an indicator with the belt color.

`/kata/kata/styled/no belt`

Shortcut for specifying `/kata/kata/styled/belt=false`

`/kata/kata/styled/belt color`

The belt color for the indicator set in the note in the margin (in case this indicator as well as the margin are enabled, see `/kata/kata/styled/margin` and `/kata/kata/styled/belt`).

`/kata/kata/styled/note`

A note which is placed in the margin at the beginning of the `kata`→P. 3. Is only placed if the margin was enabled (see `/kata/kata/styled/margin`).
Note: If the belt indicator was enabled (`/kata/kata/styled/belt`) but no belt color was set (`/kata/kata/styled/belt color`), the value of this key is used as fallback for the belt color.

2.4.5 Step environment

These options are specific to `kataStep`→P. 3.

`/kata/step/before first`

This key can be used to inject arbitrary code which is executed *before the first* `step`→P. 3.
You may use this for example to place a `\vfill` before the first step, resulting in the steps being placed closer to the pagebreak which might make (digitally) reading easier.

2.4.6 Typical content commands

Similarly like for global options there is a set of options which apply to all *typical* content commands. *Typical* here refers to content commands which draw something and also have a label. In particular `\kataDir`→P. 4 does not count as *typical* as it draw no label.

`/kata/waza/flip` (bool, initially: false, default: true)
`/kata/stance/flip` (bool, initially: false, default: true)

Flips the respective label to the other side.

`/kata/waza/style`
`/kata/stance/style`

Appends to the respective style of the component. Only adjusts the style for this single invocation (scoping).

`/kata/waza/label`
`/kata/stance/label`

Appends to the respective style for the label of the component. Only adjusts the style for this single invocation (scoping).
The style only applies to *non-flipped* labels.

`/kata/waza/label flipped`
`/kata/stance/label flipped`

Similar like `/kata/waza/label` etc.
The style only applies to *flipped* labels.

`/kata/waza/color`
`/kata/stance/color`

Sets the respective color for the component.
Under the hood this forwards to the respective key in `/kata/color/waza` → P. 8 etc.

Waza command no options apart from the common ones listed above.

Stance command no options apart from the common ones listed above.

2.4.7 Atypical content commands

Options for commands specifying content but which do not count as *typical* (see section 2.4.6).

Dir command

`/kata/dir/flip` (bool, initially: false, default: true)

Flips the arrow indicating a turn. Just specifying the angles for a turn is not enough to fully specify a turn. E.g. when turning from 0° to 180° the information whether to turn clockwise or counter-clockwise is still lacking. `\kataDir` → P. 4 will draw the arrow in one direction, it is the wrong one you can draw it in the other one with this option.
Has no effect if ⟨*angleA*⟩ is set to *center*.

Kiai command nothing additional

2.4.8 Misc commands

This section contains miscellaneous commands.

`\kataStepPhantom`

Places a phantom/invisible `step` → P. 3 which can be used to "skip" one place in the grid. This is exactly what `kataRaw` → P. 3 potentially appends to the last line in order to generate a proper grid.

Note for performance reasons, internally this does not place a `tikzpicture`. The only goal is to produce the same spacing like there would have been a `step`[→P. 3].
Also note this increases the counter which numbers the `step`[→P. 3]s. This is required because the grid generation relies on this counter. You'll notice the effect because the indicator displaying the counter will seem to have skipped one number.

2.4.9 Presets

In order to make common but more complex style modifications easier, this package defines some *presets* which can usually be applied with the respective `<> preset` key.

Presets for the border

```
/kata/preset/border/arrow up
/kata/preset/border/arrow down
/kata/preset/border/arrow right
/kata/preset/border/arrow left
```

Adjust `/kata/step/border style`[→P. 7] by appending so that the border shows two arrows. The arrows are placed on opposite sides of the rectangle both pointing in the same direction.
Which direction depends on which of the keys is chosen. The two sides showing the arrows are the following:

- edge on the *right* and *left* side for `/kata/preset/border/arrow up` and `/kata/preset/border/arrow down`
- edge on the *top* and *bottom* side for `/kata/preset/border/arrow right` and `/kata/preset/border/arrow left`

`/kata/preset/border/accent=`⟨*color*⟩

Colors the border with ⟨*color*⟩. Also fills the border with 5 % (transparent for the rest part) of that color to highlight/accent it.
As ⟨*color*⟩, everything can be passed here which Ti*k*Z can use as a color.

Presets for steps

`/kata/preset/step/align page bottom`

You usually write `kata`[→P. 3]s in two stages. First, some additional explanations. Second, the `step`[→P. 3]s. This preset sets the steps up so they are pushed down as far as possible on the first page. The idea is that this way when reading (digitally), you need to skip less space when switching from one page to the next one. There is one caveat: If you don't use sections and don't place explanations before the steps, this preset might do nothing.
Note: Under the hood, this just sets `/kata/step/before first`[→P. 10] to `\vfill`.

`/kata/preset/step/align reset`

Undoes (locally) the changes done by `/kata/preset/step/align page bottom` etc.

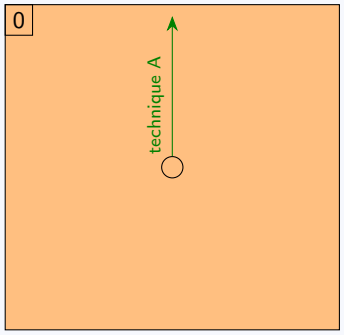
Note, usually you should not need this, as setting these keys/presets is always local (setting it for one `kata` → P. 3 won't affect the other ones).

Custom presets Note: Styles (e.g. `/kata/waza/style` → P. 11) are evaluated in a `/tikz` context eventually. So if you want to declare a custom reusable style here is an example how you could declare a preset/style which only sets a background color (frame stays black) with less transparency:

Mocks are only used here in the docs avoid problems showing this inside the example box <*doc>

Idea how to manually define a reusable style

```
\tikzset{
  myPreset/.style={fill=#1!50},
}

\kataSetup{columns=0}

\begin{kata}
  \parindent=0pt

  \begin{step}[border style={myPreset=orange}]
    \waza{technique A}{0}
  \end{step}
\end{kata}
```



</doc>

2.5 Extended Example

Note: The output looks slightly different in the example's output here. Usually, the section is a proper section and the marginline is really show in the margin.

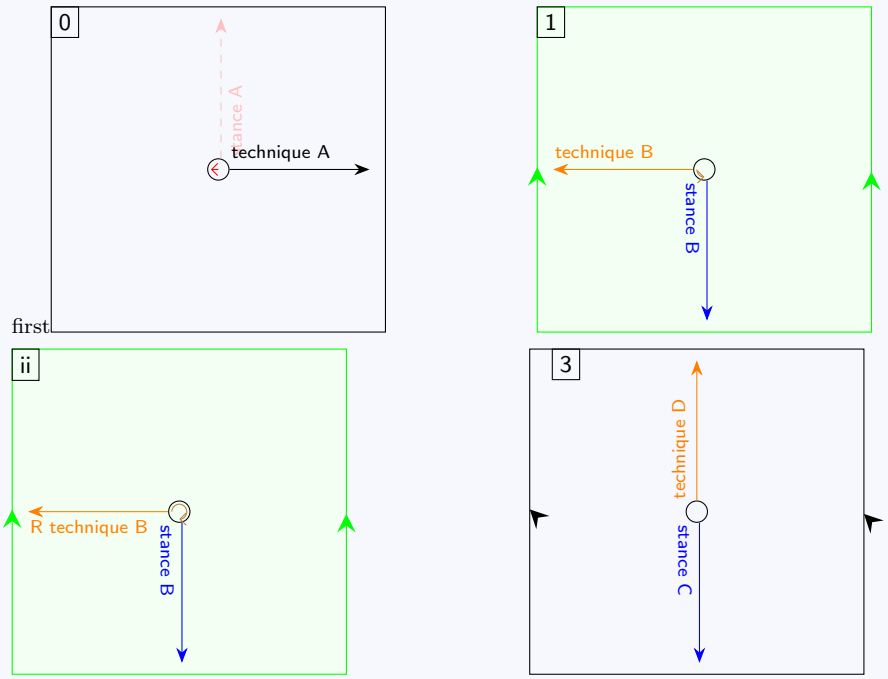
Mocks are only used here in the docs avoid problems showing this inside the example box <*doc>

Extended example with all kinds of possible customizations

```
\kataSetup{styled kata={section,margin,belt},columns=0}

\begin{kata}[columns=2,color={waza=orange},no section,no margin,belt ↩
      ↪color=orange,title={Extended Example},step={before first={first}}]
  Some explanation -- normal \LaTeX{} code
```

```
  \parindent=0pt

  \begin{step}[color={stance=pink},stance={style=dashed}]
    \stance{stance A}{0}
    \waza{technique A}{-90}[color=black]
    \dir{center}{90}
  \end{step}
  \kataSetup{color={dir=brown}}
  {
    \kataSetup{border preset={arrow up,accent=green}}
    \begin{step}[]
      \stance{stance B}{180}
      \waza{technique B}{90}
      \dir{90}{180}
    \end{step}
    \renewcommand\kataStepCnt[1]{\roman{#1}}
    \begin{step}
      \stance{stance B}{180}
      \waza[R]{technique B}{90}[flip]
      \dir[flip]{90}{180}
    \end{step}
  }
  \pgfkeys{
    /kata/style/step counter/.append style={xshift=1em},
    /kata/style/border/.append style={/kata/preset/helpers/arrowUD=45},
  }
  \begin{step}
    \stance{stance C}{180}
    \waza{technique D}{0}
  \end{step}
\end{kata}
```

Some explanation – normal LaTeX code



first

</doc>

3 Memoization

This package supports externalization via the `memoize`[→CTAN] package.

In principle, the individual pictures generated by this package are quite simple and fast to draw. However, once a document contains lots of these pictures, externalization can significantly improve the compilation time of a document. To give you numbers on that: a document with around 450 `step`[→P. 3]s took me about 15 s to compile. With the use of `memoize`[→CTAN], the compilation time reduced to about 4 s (all measurements done with LuaLaTeXand only measuring the final run). But keep in mind, externalization often leads to more compilation cycles. Thus, you need to evaluate for your individual setting whether externalization makes sense for you.

When `memoize`[→CTAN] is loaded, this package automatically sets up memoization for `step`[→P. 3]s. Still, you may need to perform a few actions (such as install a library and make `memoize`[→CTAN] use it) to make memoization work which are described in `memoize`[→CTAN]'s documentation. Also note that `memoize`[→CTAN] should always be *loaded before this package*.

3.1 Where the memoize integration reaches its limits

All styles and presets defined by this package are set up so that changes result in updates of the generated externs. But this reaches its limits when you start defining and using your own styles. For example the example given in section 2.4.9 would not lead to an update when e.g. the style `myPreset` is being modified. This can lead to confusion when modifications are not reflected in the output. It can also lead to unexpected behavior when modifying the style mid-document. For such cases you'll need to familiarize yourself with the `memoize`[→CTAN] package and learn how to make your customizations memoization-aware.

3.2 Advice when to enable memoization

Personally, I use the following strategy. Generally, I activate `memoize`[→CTAN]. While working on the document, I use the `readonly` option of *memoize*. This way, no new externs are generated and I avoid increasing compilation time by the requirement of additional compilation cycles. Still, compilation is sped up by the externs which were previously generated. Then, after work is finished, I compile without the `readonly` option to generate the externs for next time.

Note, `memoize`[→CTAN] recommends disabling it when generating the final version of the document or more generally, a version which gets released.

4 Implementation

```
\RequirePackage{pgfkeys}
\RequirePackage{pgfopts}
\RequirePackage{coloredbelts}
\RequirePackage{tikz}
\usetikzlibrary{calc,positioning,fit,arrows.meta}
```

```
\usetikzlibrary{decorations.markings,decorations.pathmorphing,backgrounds}
```

Define marginline in case it isn't already defined (so the package works without KOMA-classes as well).

```
\AddToHook {begindocument} [kata/provideMarginline] {%
  \providecommand\marginline[1]{%
    \marginpar[{\raggedleft #1}]{\raggedright #1}%
  }%
}
```

Define own string compare for equality so the package works without KOMA as well.

```
\ExpandArgs {nnc} \UseName {cs_set_eq:NN} {\kata@ifstr} {str_if_eq:eeTF}
```

4.1 General pgfkeys setup

```
\pgfkeys{
```

for package wide settings

```
  /kata/setup/.is family,
```

for settings regarding for the kataRaw env

```
  /kata/kata/.is family,
```

for settings specific for the wrapper around kataRaw (kata)

```
  /kata/kata/styled/.is family,
```

for settings regarding for the step env

```
  /kata/step/.is family,
```

for settings regarding for the stance command

```
  /kata/stance/.is family,
```

for settings regarding for the waza command

```
  /kata/waza/.is family,
```

for settings regarding for the dir command

```
  /kata/dir/.is family,
```

for settings regarding coloring

```
  /kata/color/.is family,
```

internal style definitions

```
  /kata/style/.is family,
```

presets for better usability

```
  /kata/preset/border/.is family,
  /kata/preset/step/.is family,
```

forwards in order to DRY

```
  /kata/forward/.is family,
}
```

4.2 Set up pgfkeys-forwards

```
\pgfkeys{
  /kata/forward,
```

Common keys which simply delegate to other keys

```
  % only "forwards" -> adds not to memoize context
  delegate common/.style={
    stance/.code = \pgfkeys{/kata/stance,##1},
    waza/.code   = \pgfkeys{/kata/waza,##1},
    dir/.code    = \pgfkeys{/kata/dir,##1},
```

```
      color/.code  = \pgfkeys{/kata/color,##1},
      step/.code   = \pgfkeys{/kata/step,##1},
    },
```

Common keys which append to styles

```
    % only appends -> adds not to memoize context
    append common/.style={
      head style/.style = {/kata/style/head/.append style = {##1}},
      border style/.style = {/kata/style/border/.append style = {##1}},
      border preset/.forward to = /kata/style/border preset,
      step preset/.forward to = /kata/style/step preset,
      step counter style/.style = {/kata/style/step counter/.append style = {##1}},
    },
  }
  %
  %
  % \subsection{Package wide options}
  % Defer keys in setup to the respective locations
  %    \begin{macrocode}
  \pgfkeys{
    /kata/setup,
    % only "forwards" -> adds not to memoize context
    columns/.style = {/kata/kata,columns={#1}},
    % not relevant for step -> adds not to memoize context
    definealiases/.code = {%
      \AddToHook{begindocument}[kata/aliases]{%
        \kataAliases%
      }%
    },
    styled kata/.code = \pgfkeys{/kata/kata/styled,#1},
    %
    /kata/forward/delegate common,
    /kata/forward/append common,
  }
```

Setup package option parsing

```
  \ProcessPgfPackageOptions{/kata/setup}
```

A setup command for setting package options later

\kataSetup

```
  \NewDocumentCommand{\kataSetup}{m}{%
    \pgfkeys{/kata/setup,#1}%
    \ignorespaces%
  }%
```

4.3 Optionally define command aliases

\kataAliases

```
  \newif\ifkata@aliasesdefined
  \kata@aliasesdefinedfalse
  \NewDocumentCommand{\kataAliases}{}{
    \ifkata@aliasesdefined\else%
      \kata@aliasesdefinedtrue%
      \newcommand{\kata@warningdefined}[1]{\PackageWarning{kata}{Macro ##1 is already defined, di
      \@ifundefined{stance}{\let\stance\kataStance}{\kata@warningdefined{\string\stance}}%
```

```
    \@ifundefined{waza}{\let\waza\kataWaza}{\kata@warningdefined{\string\waza}}%
    \@ifundefined{dir}{\let\dir\kataDir}{\kata@warningdefined{\string\dir}}%
    \@ifundefined{kiai}{\let\kiai\kataKiai}{\kata@warningdefined{\string\kiai}}%
    % environment begin/end macros
    \@ifundefined{step}{%
      \let\step\kataStep%
      \let\endstep\endkataStep%
    }{%
      \PackageWarning{kata}{Environment/Macro \string\step is already defined, did not define t
    }%
  \fi%
}
```

4.4 Core internals

A counter for each pictogram to enable referencing.

```
\newcounter{kata@kata}
\newcounter{kata@step}[kata@kata]
```

`\kata@requirestep` enforces something is run within the step environment (helper)

```
\newif\ifkata@instep
\newcommand*{\kata@requirestep}[1]{%
  \ifkata@instep\else
    \PackageError{kata}{#1 used outside step environment}{}
  \fi
}
```

`\kata@requirekata` enforces something is run within the kata environment (helper)

```
\newif\ifkata@inkata
\newcommand*{\kata@requirekata}[1]{%
  \ifkata@inkata\else
    \PackageError{kata}{#1 used outside kata environment}{}
  \fi
}

\newif\ifkata@stance@flip
\newif\ifkata@waza@flip
\newif\ifkata@dir@flip
%
\newif\ifkata@show@section
\newif\ifkata@show@margin
\newif\ifkata@show@belt
%
\newcount\kata@tmpcnta
```

4.5 The kataRaw environment

This is the low-level raw kata environment. It is only for wrapping the `step`s and trying to generate a proper grid (by placing the correct space in the end).

```
\pgfkeys{
  /kata/kata,
  % not relevant for step -> adds not to memoize context
  columns/.store in = \kata@columns,
```

```
    columns = 6,
    columns/.value required,
    %
    /kata/forward/delegate common,
    /kata/forward/append common,
  }
```

`kataRaw` (*env.*) this environment is basically a container for a sequence of steps. Arguments:

1. optional: keys from `/kata/kata`

```
  \NewDocumentEnvironment{kataRaw}{O{}}{%
    \pgfkeys{/kata/kata,#1}%
    \setlength{\lineskip}{6pt}%
    \stepcounter{kata@kata}%
    \kata@inkatatrue%
```

there is no harm in a single word/pictogram in the last line to turn off the corresponding penalty

```
    \widowpenalty=0
    \ignorespaces%
  }{%
```

add hidden steps to the last line. This ensures a uniform looking grid of pictograms

```
    \ifnum\kata@columns=0\else%
      \pgfmathtruncatemacro{\kata@missing}{mod(\kata@columns - mod(\value{kata@step},\kata@colum
      \ifnum\kata@missing>0
        \foreach \i in {1,...,\kata@missing} {%
          \kataStepPhantom%
        }%
      \fi%
    \fi%
```

avoid left-aligning the last line. Instead it should be justified in order to obtain a proper grid

```
    \parfillskip=0pt
    \par%
  }
```

4.6 The kata environment

`\kataSection`

```
  \newcommand\kataSection[1]{\section{#1}}

  \pgfkeys{
    /kata/kata/styled,
    /kata/kata/styled/.search also={/kata/kata},
    % not relevant for step -> adds not to memoize context
    section/.is if = kata@show@section,
    section/.default = true,
    section = false,
    no section/.style={/kata/kata/styled/section=false},
    %
    title/.store in = \kata@title,
    title = {},
```

```
  title/.value required,
  %
  %
  margin/.is if = kata@show@margin,
  margin/.default = true,
  margin = false,
  no margin/.style={/kata/kata/styled/margin=false},
  %
  belt/.is if = kata@show@belt,
  belt/.default = true,
  belt = false,
  no belt/.style={/kata/kata/styled/belt=false},
  %
  belt color/.store in = \kata@belt,
  belt color = {},
  belt color/.value required,
  %
  note/.store in = \kata@note,
  note = {},
  note/.value required,
}
```

`kata` (*env.*) this environment is basically a container for a sequence of steps. TODO Arguments:

1. optional: keys from `/kata/kata`

```
\NewDocumentEnvironment{kata}{O{}}{%
  \pgfkeys{/kata/kata/styled,#1}%
  \ifkata@show@section%
    \kataSection{\kata@title}%
  \fi%
  \ifkata@show@margin%
    \ifkata@show@belt%
      \ifx\kata@belt\@empty%
        \kata@beltmargin{\kata@note}{\kata@note}%
      \else%
        \kata@beltmargin{\kata@note}{\kata@belt}%
      \fi%
    \else%
      \kata@beltmargin{\kata@note}{}%
    \fi%
  \fi%
  \begin{kataRaw}%
  \ignorespaces%
}{%
  \end{kataRaw}%
}
```

`\kata@beltmargin@raw` Takes two arguments:

1. the note
2. the belt color

If an argument is empty, the corresponding element will not be shown.

```
\newcommand{\kata@beltmargin@raw}[2]{%
  \marginline{%
    \centering\footnotesize%
    \kata@ifstr{#1}{}{}{%
      $\downarrow$ #1%
    }%
```

only linebreak if both are non-empty

```
  \kata@ifstr{#1}{}{}{\kata@ifstr{#2}{}{}{\\}}%
  \kata@ifstr{#2}{}{}{%
      \ColorBelt*[scale=0.1]{#2}%
  }%
  }%
}
```

`\kata@beltmargin` Analog to `\kata@beltmargin@raw` but wraps for display policy reasons: If everything is empty this avoids drawing a marginline at all.

```
\newcommand{\kata@beltmargin}[2]{%
  \kata@ifstr{#1}{}{%
    \kata@ifstr{#2}{}{}{%
      \kata@beltmargin@raw{#1}{#2}%
    }%
  }{%
    \kata@beltmargin@raw{#1}{#2}%
  }%
}
```

4.7 The step environment

```
\pgfkeys{
  /kata/step,
  % not relevant for inner step -> adds not to memoize context
  before first/.store in = \kata@step@first@before,
  before first = {},
  before first/.value required,
  %
  % mostly "forwards" -> adds not to memoize context
  /kata/forward/delegate common,
  /kata/forward/append common,
}
```

`\kata@step@interspace` places a space (with glue) / a linebreak to be used between two `step`s What exactly is placed is determined based on two parameters:

1. the amount of columns the resulting grid should have
2. the index (starting at 0) of the current `step`

This macro assumes the space is placed *before* the `step`.

```
\newcommand{\kata@step@interspace}[2]{%
  \ifnum#1=0%
    \hskip 0.0pt plus 1.0fil %
  \else%
    \pgfmathtruncatemacro{\kata@step@column}{mod(#2,#1)}%
```

```
    \ifnum\kata@step@column=#1%
      \par
    \else%
      \hskip 0.0pt plus 1.0fil %
    \fi%
  \fi%
}%
```

`\kataStepPhantom` Places a phantom (invisible) `step` with the correct width and spacing to be consistent with the grid.

```
\NewDocumentCommand{\kataStepPhantom}{}{%
  \kata@step@interspace{\kata@columns}{\value{kata@step}}%
  \rule{\the\kata@phantomstep@width}{\the\kata@phantomsteprule@width}
  \refstepcounter{kata@step}
}
```

`step` (*env.*) this environment always should wrap `kata@step`. It handles the aspects of positioning the pictogram. Arguments: 1 (optional): pgfkeys

```
\NewDocumentEnvironment{kataStep}{O{}+b}{%
  \kata@requirekata{\string\kataStep}%
  \pgfkeys{/kata/step,#1}%
  \ifnum\value{kata@step}=0
    \kata@step@first@before\noindent%
  \else%
    \kata@step@interspace{\kata@columns}{\value{kata@step}}%
  \fi%
  \begin{kata@step}%
    #2
  \end{kata@step}%
  \refstepcounter{kata@step}
}{}
```

`kata@step` (*env.*) the environment to draw everything else in

```
\newenvironment*{kata@step}{%
  \kata@requirekata{\string\step}%
  \begin{tikzpicture}[font=\scriptsize]%
    \begin{scope}[local bounding box=output]%
      \kata@head\kata@nothing%
      \kata@insteptrue%
}{
    \end{scope}%
```

Drawing the border on the background has the advantage the rectangle can be filled and colorize the background as well.

```
    \begin{scope}[on background layer]%
      \node[/kata/style/border] (t1) {};%
      \node[/kata/style/step counter] (cnt) {\kataStepCnt{kata@step}};%
    \end{scope}%
  \end{tikzpicture}%
}
```

4.8 Specify the Content

`\kataKiai` draw indicator for *kiai*. Arguments:

1. (optional) distance to head TODO: not implemented yet (probably will become pgf-interface)
2. angle

```
\NewDocumentCommand{\kataKiai}{O{}m}{
  \kata@requirestep{\string\kataKiai}
  \def\base{#2+90} \def\off{30} \def\ext{40} \def\roff{4pt} \def\len{0.75}
  \draw[shorten <=10pt,red]
  ($(head)+(\base-\off:\roff)$) edge[decorate,decoration={bent,aspect=0.3,amplitude=-2.5pt}] co
  ($(head)+(\base+\off:\roff)$) edge[decorate,decoration={bent,aspect=0.3,amplitude=+2.5pt}] co
  \path (tmpA) -- (tmpB) node[midway,sloped,font={\sffamily\scriptsize},red,draw,decorate,decor
}
```

4.8.1 The stance command

```
\pgfkeys{
  /kata/stance,
  % only inside step -> contained in hash of the body anyhow -> adds not to memoize context
  flip/.is if = kata@stance@flip,
  flip/.default = true,
  style/.style = {/kata/style/stance/.append style = {#1}},
  label/.style = {/kata/style/stance label/.append style = {#1}},
  label flipped/.style = {/kata/style/stance label flipped/.append style = {#1}},
  color/.forward to = /kata/color/stance,
}
```

`\kataStance` draw the arrow/path showing the stance Usage:

1. (optional) side to use (L/R) / prefix
2. name/text
3. angle
4. (optional) pgfkeys interface

```
\NewDocumentCommand{\kataStance}{o+mmO{}}{%
  {%
    \pgfkeys{/kata/stance,#4}%
    \kata@requirestep{\string\kataStance}%
    \pgfmathsetmacro{\kata@dir}{#3+90}%
    \pgfmathsetmacro{\x}{abs(sin(\kata@dir))}%
    \pgfmathsetmacro{\y}{-abs(cos(\kata@dir))}%
    \IfNoValueTF{#1}{\def\kata@nodecontent{}}{\def\kata@nodecontent{#1 }}%
    \draw[/kata/style/stance={\x}{\y}]
    (head) -- ++(\kata@dir:2)
    node[\ifkata@stance@flip/kata/style/stance label flipped\else/kata/style/stance label\fi]
    {\hangafter=1\settowidth\hangindent{\kata@nodecontent}\kata@nodecontent#2}
    ;
  }%
}
```

4.8.2 The waza command

```
\pgfkeys{
```

```
  /kata/waza,
  % only inside step -> contained in hash of the body anyhow -> adds not to memoize context
  flip/.is if = kata@waza@flip,
  flip/.default = true,
  style/.style = {/kata/style/waza/.append style = {#1}},
  label/.style = {/kata/style/waza label/.append style = {#1}},
  label flipped/.style = {/kata/style/waza label flipped/.append style = {#1}},
  color/.forward to = /kata/color/waza,
}
```

`\kataWaza` draw the arrow/path showing a technique. Arguments:

1. (optional) side to use (L/R) / prefix
2. name/text
3. angle
4. (optional) pgfkeys interface

```
\NewDocumentCommand{\kataWaza}{o+mmO{}}{%
  {%
    \kata@requirestep{\string\kataWaza}%
    \pgfkeys{/kata/waza,#4}%
    \IfNoValueTF{#1}{\def\kata@nodecontent{}}{\def\kata@nodecontent{#1 }}%
    \pgfmathsetmacro{\kata@dir}{#3+90}%
    \draw[/kata/style/waza]
    (head) -- ++(\kata@dir:2)
    node[\ifkata@waza@flip/kata/style/waza label flipped\else/kata/style/waza label\fi]
    {\hangafter=1\settowidth\hangindent{\kata@nodecontent}\kata@nodecontent#2}
    ;
  }%
}
```

4.8.3 The dir command

```
\pgfkeys{
  /kata/dir,
  % only inside step -> contained in hash of the body anyhow -> adds not to memoize context
  flip/.is if = kata@dir@flip,
  flip/.default = true,
  style/.style = {/kata/style/dir/.append style = {#1}},
  color/.forward to = /kata/color/dir,
}
```

`\kataDir` draw a short arrow/path indicating the next turn/direction. Arguments:

1. (optional) flip / pgfkeys
2. source (Center,angle)
3. dst (angle)

```
\NewDocumentCommand{\kataDir}{O{}mm}{
  {%
    \kata@requirestep{\string\kataDir}
    \pgfkeys{/kata/dir,#1}
```

```
    \pgfmathsetmacro{\kata@out}{#3+90}
    \kata@ifstr{#2}{center}{
      \draw[/kata/style/dir] (head.center) -- ++(\kata@out:.33em);
    }{
      \pgfmathsetmacro{\kata@in}{#2+90}
      \pgfmathsetcount{\kata@tmpcnta}{\kata@out-\kata@in}
      \ifkata@dir@flip
        \ifnum\kata@tmpcnta<0
          \pgfmathsetmacro{\kata@outdeltaii}{360+\the\kata@tmpcnta}
        \else
          \pgfmathsetmacro{\kata@outdeltaii}{\the\kata@tmpcnta-360}
        \fi
      \else
        \pgfmathsetmacro{\kata@outdeltaii}{\the\kata@tmpcnta}
      \fi
      \draw[/kata/style/dir]
      (head.center) ++ (\kata@in:.33em)
      arc[start angle=\kata@in,delta angle=\kata@outdeltaii,radius=.33em]
      ;
    }
  }%
}
```

4.8.4 Internal elements

`\kata@nothing` ensure the box is always at least the same size (internal command + always placed)

```
\newcommand*{\kata@nothing}{
  \draw[draw=none] (head) -- ++(0:2.15) -- ++(180:4.3) -- ++(90:2.15) -- ++(-90:4.3);
}
```

`\kata@head` draw the circle symbolizing the head (internal command + always placed)

```
\newcommand*{\kata@head}{
  \node[/kata/style/head] (head) {};
}
```

4.9 Color keys

```
\pgfkeys{
  % csnames need to be (and are) added to memoize context when setting up memoize
  /kata/color,
  %
  stance/.store in = \kata@color@stance,
  stance/.value required,
  stance = blue,
  %
  waza/.store in = \kata@color@waza,
  waza/.value required,
  waza = green!50!black,
  %
  dir/.store in = \kata@color@dir,
  dir/.value required,
  dir = red,
  %
  head/.store in = \kata@color@head,
```

```
  head/.value required,
  head = black,
  %
  border/.store in = \kata@color@border,
  border/.value required,
  border = black,
  %
  step counter/.store in = \kata@color@stepcounter,
  step counter/.value required,
  step counter = black,
}
```

4.10 Styles

4.10.1 Styles for drawn elements

```
\pgfkeys{
  /kata/style,
  %
  head/.style = {
    /kata/style/head/.meaning to context,
    circle,
    draw=\kata@color@head,
    inner xsep=0.333em,
  },
  %
  stance/.style 2 args = {
    /kata/style/stance/.meaning to context,
    draw=\kata@color@stance,
    -{Stealth[scale=1.2]},
    transform canvas={xshift=#1,yshift=#2},
  },
  %
  waza/.style = {
    /kata/style/waza/.meaning to context,
    draw=\kata@color@waza,
    -{Stealth[scale=1.2]},
  },
  %
  dir/.style = {
    /kata/style/dir/.meaning to context,
    draw=\kata@color@dir,
    -{Straight Barb},
  },
  %
  border/.style = {
    /kata/style/border/.meaning to context,
    rectangle,
    draw=\kata@color@border,
    fit=(output),
    inner sep=0pt,
    outer sep=0pt,
    inner xsep=0.2em,
  },
  %
```

```
  step counter/.style = {
    /kata/style/step counter/.meaning to context,
    draw=\kata@color@stepcounter,
    font=\sffamily\small,
    outer sep=0pt,
    at={(t1.north west)},
    below right,
    rectangle,
  },
  %
  % only forwards -> adds not to memoize context
  border preset/.code={
    \pgfkeysalso{/kata/preset/border,#1}%
  },
  %
  % only forwards -> adds not to memoize context
  step preset/.code={
    \pgfkeysalso{/kata/preset/step,#1}%
  },
}
```

4.10.2 Styles for labels

```
\pgfkeys{
  /kata/style,
  %
  stance label base/.style = {
    /kata/style/stance label base/.meaning to context,
    \kata@color@stance,
    midway,
    sloped,
    font=\sffamily\scriptsize,
    text width=1.8cm,
  },
  stance label/.style = {
    /kata/style/stance label/.meaning to context,
    /kata/style/stance label base,
    below,
  },
  %
  stance label flipped/.style = {
    /kata/style/stance label flipped/.meaning to context,
    /kata/style/stance label base,
    above,
  },
  %
  waza label base/.style = {
    /kata/style/waza label base/.meaning to context,
    \kata@color@waza,
    midway,
    sloped,
    font=\sffamily\scriptsize,
    text width=1.8cm,
  },
  %
```

```
  waza label/.style = {
    /kata/style/waza label/.meaning to context,
    /kata/style/waza label base,
    above,
  },
  %
  waza label flipped/.style = {
    /kata/style/waza label flipped/.meaning to context,
    /kata/style/waza label base,
    below,
  },
}
```

4.10.3 Presets for styles

Helpers

```
\pgfkeys{
  % is not used directly -> adds not to memoize context
  /kata/preset/helpers/.cd,
  arrowRL/.style={
    postaction={decorate},decoration={
      markings,
      mark=at position 0.125
        with {\pgftransformrotate{#1 - 270}\arrow[line width=2pt]{stealth}},
      mark=at position 0.625
        with {\pgftransformrotate{#1 - 270}\arrowreversed[line width=2pt]{stealth}},
    },
  },
  % is not used directly -> adds not to memoize context
  arrowUD/.style={
    postaction={decorate},decoration={
      markings,
      mark=at position 0.375
        with {\pgftransformrotate{#1}\arrow[line width=2pt]{stealth}},
      mark=at position 0.875
        with {\pgftransformrotate{#1}\arrowreversed[line width=2pt]{stealth}},
    },
  },
}
```

Border presets

```
\pgfkeys{
  /kata/preset/border,
  % only appends -> adds not to memoize context
  arrow right/.style={
    /kata/style/border/.append style={
      /kata/preset/helpers/arrowRL=-90,
    },
  },
  % only appends -> adds not to memoize context
  arrow left/.style={
    /kata/style/border/.append style={
      /kata/preset/helpers/arrowRL=+90,
    },
```

```
    },
    % only appends -> adds not to memoize context
    arrow up/.style={
      /kata/style/border/.append style={
        /kata/preset/helpers/arrowUD=0,
      },
    },
    % only appends -> adds not to memoize context
    arrow down/.style={
      /kata/style/border/.append style={
        /kata/preset/helpers/arrowUD=180,
      },
    },
    % only appends -> adds not to memoize context
    accent/.style={
      /kata/style/border/.append style={
        fill=#1!5,
        draw=#1,
      },
    },
  }
```

Step presets

```
  \pgfkeys{
    /kata/preset/step,
    % only appends -> adds not to memoize context
    align page bottom/.style={
      /kata/step/before first={\vfill},
    },
    align reset/.style={
      /kata/step/before first={},
    },
  }
```

4.11 Misc

`\kataStepCnt` Formats the counter for the step indicator

```
  \NewDocumentCommand{\kataStepCnt}{m}{\arabic{#1}}
```

4.12 memoize integration

```
  \@ifpackageloaded{memoize}{%
    \mmzset{%
      auto={kata@step}{
        memoize,
        context={stepCnt->\arabic{kata@step},kata version->\kata@version,},
        csname meaning to context={kataStepCnt code},
        csname meaning to context={kata@color@stance},
        csname meaning to context={kata@color@waza},
        csname meaning to context={kata@color@dir},
        csname meaning to context={kata@color@head},
        csname meaning to context={kata@color@border},
        csname meaning to context={kata@color@stepcounter},
```

```
    },
  }%
}{%
  \RequirePackage{memoizable}
}%
```

4.13 Internal variables / lengths maybe useful for debugging

\kata@phantomsteprule@width

```
\newdimen\kata@phantomsteprule@width
\kata@phantomsteprule@width=0pt
```

\kata@phantomstep@width

```
\newdimen\kata@phantomstep@width
```

A word on the calculation:

- `4.3cm` this is the forced width of each step
- `0.2em * 2` this is the xsep which is present on both sides (left+right)
- `0.4pt * 2 / 2` this is the (default) border linewidth which is present on both sides (left+right), but half of the border presses to the inside (thus the division by 2)

See how this may be fragile in terms of changes to how the step is presented (adjustments regarding the `xsep` or the linewidth of the border

```
\kata@phantomstep@width=\dimexpr 4.3cm + 0.2em * 2 + 0.4pt * 2 / 2\relax
```

5 Change History

0.1.0

General: Converted to DTX file . . 1

6 Index

Numbers written in italic refer to the page where the corresponding entry is described; numbers underlined refer to the definition; numbers in roman refer to the pages where the entry is used.

A

accent key 12
align page bottom key 12
align reset key 13
arrow down key 12
arrow left key 12
arrow right key 12
arrow up key 12

B

\base 23
before first key 10
belt key 10
belt color key 10
border key 8

C

columns key 8, 9

D

definealiases key 9
dir key 8

\dir 18

E

environments:
- kata 20
- kata@step 22
- kataRaw 19
- step 22

\ext 23

F

flip key 11

H

head key 8

I

\ifkata@aliasesdefined 18
\ifkata@dir@flip 18, 25
\ifkata@inkata 18
\ifkata@instep 18
\ifkata@show@belt 19, 20
\ifkata@show@margin 18, 20
\ifkata@show@section 18, 20
\ifkata@stance@flip 18, 24
\ifkata@waza@flip 18, 24

K

kata (env.) 20
kata/border preset key 7
kata/border style key 7
kata/color key 7
kata/dir key 7
kata/head style key 7
kata/stance key 7
kata/step counter style key 8
kata/step preset key 8
kata/waza key 7
\kata@aliasesdefinedfalse 18
\kata@aliasesdefinedtrue 18
\kata@belt 20
\kata@beltmargin 20, 21
\kata@beltmargin@raw 21, 21
\kata@color@border 26, 27
\kata@color@dir 26
\kata@color@head 26
\kata@color@stance 25–27
\kata@color@stepcounter 26, 27
\kata@color@waza 26, 28
\kata@columns 19, 22
\kata@dir 23, 24
\kata@head 22, 25
\kata@ifstr 16, 21, 25
\kata@in 25
\kata@inkatatrue 19
\kata@insteptrue 22
\kata@missing 19
\kata@nodecontent 23, 24
\kata@note 20
\kata@nothing 22, 25
\kata@out 25
\kata@outdeltaii 25
\kata@phantomstep@width 22, 30
\kata@phantomsteprule@width . . 22, 30
\kata@requirekata 18, 22
\kata@requirestep 18, 23–25
kata@step (env.) 22
\kata@step@column 22
\kata@step@first@before 21, 22
\kata@step@interspace 21, 22
\kata@title 20
\kata@tmpcnta 19, 25
\kata@version 30
\kata@warningdefined 18
\kataAliases 17, 18
\kataDir 18, 24
\kataKiai 18, 23
kataRaw (env.) 19
\kataSection 19, 20
\kataSetup 17
\kataStance 18, 23
\kataStep 18, 22
\kataStepCnt 23, 29
\kataStepPhantom 19, 22
\kataWaza 18, 24

Keys
- /kata/
  - kata/border preset 7
  - kata/border style 7
  - kata/color 7
  - kata/dir 7
  - kata/head style 7
  - kata/stance 7
  - kata/step counter style 8
  - kata/step preset 8
  - kata/waza 7
  - setup/border preset 7
  - setup/border style 7
  - setup/color 7
  - setup/dir 7
  - setup/head style 7
  - setup/stance 7
  - setup/step counter style . . . 8
  - setup/step preset 8
  - setup/waza 7
  - stance/color 11
  - stance/flip 11
  - stance/label 11
  - stance/label flipped 11

stance/style 11
step/border preset 7
step/border style 7
step/color 7
step/dir 7
step/head style 7
step/stance 7
step/step counter style 8
step/step preset 8
step/waza 7
waza/color 11
waza/flip 11
waza/label 11
waza/label flipped 11
waza/style 11
/kata/color/
border 8
dir 8
head 8
stance 8
step counter 8
waza 8
/kata/dir/
flip 11
/kata/kata/
columns 9
/kata/kata/styled/
belt 10
belt color 10
margin 10
no belt 10
no margin 10
no section 9
note 10
section 9
title 9
/kata/preset/border/
accent 12
arrow down 12
arrow left 12
arrow right 12
arrow up 12
/kata/preset/step/
align page bottom 12
align reset 13
/kata/setup/
columns 8
definealiases 9
styled kata 9
/kata/step/
before first 10
\kiai 18

L

\len 23

M

margin key 10
\marginline 16, 21

N

no belt key 10
no margin key 10
no section key 9
note key 10

O

\off 23

R

\roff 23

S

section key 9
setup/border preset key 7
setup/border style key 7
setup/color key 7
setup/dir key 7
setup/head style key 7
setup/stance key 7
setup/step counter style key 8
setup/step preset key 8
setup/waza key 7
stance key 8
stance/color key 11
stance/flip key 11
stance/label key 11
stance/label flipped key 11
stance/style key 11
step (env.) 22
step counter key 8
step/border preset key 7
step/border style key 7
step/color key 7
step/dir key 7
step/head style key 7
step/stance key 7
step/step counter style key 8
step/step preset key 8
step/waza key 7
styled kata key 9

T

title key 9

V

\vfill 29

W

waza key 8
\waza 18
waza/color key 11
waza/flip key 11
waza/label key 11
waza/label flipped key 11
waza/style key 11